

Deutsch–Jozsa & Qiskit

Quantum Algorithms · Lecture 3

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$$

By the end of the hour: one oracle query that beats every classical algorithm,
and the tool to run it on real hardware.

Plan

1. Recap: last lecture's reversible tricks

Plan

1. Recap: last lecture's reversible tricks
2. Oracles, properly: why the XOR form, phase kickback in general

Plan

1. Recap: last lecture's reversible tricks
2. Oracles, properly: why the XOR form, phase kickback in general
3. Deutsch's algorithm: one query beats two

Plan

1. Recap: last lecture's reversible tricks
2. Oracles, properly: why the XOR form, phase kickback in general
3. Deutsch's algorithm: one query beats two
4. The Hadamard transform: the tool Deutsch–Jozsa needs

Plan

1. Recap: last lecture's reversible tricks
2. Oracles, properly: why the XOR form, phase kickback in general
3. Deutsch's algorithm: one query beats two
4. The Hadamard transform: the tool Deutsch–Jozsa needs
5. Deutsch–Jozsa: one query beats exponentially many

Plan

1. Recap: last lecture's reversible tricks
2. Oracles, properly: why the XOR form, phase kickback in general
3. Deutsch's algorithm: one query beats two
4. The Hadamard transform: the tool Deutsch–Jozsa needs
5. Deutsch–Jozsa: one query beats exponentially many
6. Qiskit: setup, and the library

Plan

1. Recap: last lecture's reversible tricks
2. Oracles, properly: why the XOR form, phase kickback in general
3. Deutsch's algorithm: one query beats two
4. The Hadamard transform: the tool Deutsch–Jozsa needs
5. Deutsch–Jozsa: one query beats exponentially many
6. Qiskit: setup, and the library
7. The payoff: building and running Deutsch–Jozsa

Recap, in one slide

- Every classical circuit sits inside the quantum ones; cost there was gate count.

Recap, in one slide

- Every classical circuit sits inside the quantum ones; cost there was gate count.
- Controlled unitaries plus one ancilla make NOT, AND, OR reversible: XOR the result in, don't overwrite.

Recap, in one slide

- Every classical circuit sits inside the quantum ones; cost there was gate count.
- Controlled unitaries plus one ancilla make NOT, AND, OR reversible: XOR the result in, don't overwrite.
- Superposition and relative phase are the new ingredients; H twice showed phase is for cancellation.

Recap, in one slide

- Every classical circuit sits inside the quantum ones; cost there was gate count.
- Controlled unitaries plus one ancilla make NOT, AND, OR reversible: XOR the result in, don't overwrite.
- Superposition and relative phase are the new ingredients; H twice showed phase is for cancellation.
- No unitary can clone a qubit.

Recap, in one slide

- Every classical circuit sits inside the quantum ones; cost there was gate count.
- Controlled unitaries plus one ancilla make NOT, AND, OR reversible: XOR the result in, don't overwrite.
- Superposition and relative phase are the new ingredients; H twice showed phase is for cancellation.
- No unitary can clone a qubit.

Today: turn that XOR trick into a black-box oracle, then point two algorithms at it.

Part 1

Oracles, Properly

Building U_f

Two algorithms today both need a black-box function turned into a gate. That's not automatic.

- **Why not just** $|x\rangle \mapsto |f(x)\rangle$? Not reversible in general: a constant f sends many inputs to one output. Many-to-one is not a bijection, so it's not unitary.

Building U_f

Two algorithms today both need a black-box function turned into a gate. That's not automatic.

- **Why not just** $|x\rangle \mapsto |f(x)\rangle$? Not reversible in general: a constant f sends many inputs to one output. Many-to-one is not a bijection, so it's not unitary.
- **The fix:** same trick as Lecture 2's ancilla-based reversible AND/OR: XOR the result in, don't overwrite:

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$$

Building U_f

Two algorithms today both need a black-box function turned into a gate. That's not automatic.

- **Why not just** $|x\rangle \mapsto |f(x)\rangle$? Not reversible in general: a constant f sends many inputs to one output. Many-to-one is not a bijection, so it's not unitary.
- **The fix:** same trick as Lecture 2's ancilla-based reversible AND/OR: XOR the result in, don't overwrite:

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$$

- **Unitary for free.** Apply U_f twice:

$$|x\rangle |y\rangle \mapsto |x\rangle |y \oplus f(x)\rangle \mapsto |x\rangle |y \oplus f(x) \oplus f(x)\rangle = |x\rangle |y\rangle$$

$U_f^2 = I$ for every f : self-inverse $\Rightarrow$ bijective $\Rightarrow$ a permutation matrix $\Rightarrow$ unitary.
No case-by-case check needed.

Phase kickback, in general

For *any* f and *any* x , prepare the target as $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$:

$$U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle$$

The target's computational-basis value never changes; it absorbs $f(x)$ as a global phase on the query register instead.

Phase kickback, in general

For *any* f and *any* x , prepare the target as $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$:

$$U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle$$

The target's computational-basis value never changes; it absorbs $f(x)$ as a global phase on the query register instead.

Cost model. “One query” counts one call to U_f as a single unit, however large f is inside: the *query model*, distinct from the *circuit model* (gate count) used for classical circuits.

Phase kickback, in general

For *any* f and *any* x , prepare the target as $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$:

$$U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle$$

The target's computational-basis value never changes; it absorbs $f(x)$ as a global phase on the query register instead.

Cost model. “One query” counts one call to U_f as a single unit, however large f is inside: the *query model*, distinct from the *circuit model* (gate count) used for classical circuits.

This is the only fact we need. Deutsch's algorithm and Deutsch–Jozsa, up next, are both just this identity plus Hadamards on either side, nothing more to derive.

Part 2

Deutsch's Algorithm

The problem

$f : \{0, 1\} \rightarrow \{0, 1\}$ is given as a black box. Decide whether f is

- **constant** ($f(0) = f(1)$), or
- **balanced** ($f(0) \neq f(1)$),

using as few calls to f as possible.

The problem

$f : \{0, 1\} \rightarrow \{0, 1\}$ is given as a black box. Decide whether f is

- **constant** ($f(0) = f(1)$), or
- **balanced** ($f(0) \neq f(1)$),

using as few calls to f as possible.

- **Classically**: one query never suffices: seeing $f(0)$ alone is consistent with both. Two queries always suffice. Cost: **2**.

The problem

$f : \{0, 1\} \rightarrow \{0, 1\}$ is given as a black box. Decide whether f is

- **constant** ($f(0) = f(1)$), or
- **balanced** ($f(0) \neq f(1)$),

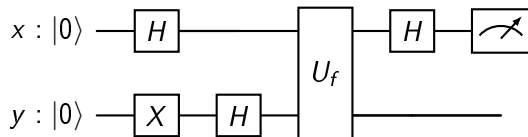
using as few calls to f as possible.

- **Classically**: one query never suffices: seeing $f(0)$ alone is consistent with both. Two queries always suffice. Cost: **2**.
- **Quantumly**: one query suffices, with certainty. Cost: **1**.

The oracle and the circuit

U_f on a query qubit x and a target qubit y , built and justified above:

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$$



Why it works

Write $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$. After the first two Hadamards, held ready for the oracle:

$$H|0\rangle \otimes H|1\rangle = \frac{1}{\sqrt{2}} \sum_x |x\rangle \otimes |-\rangle$$

Why it works

Write $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$. After the first two Hadamards, held ready for the oracle:

$$H|0\rangle \otimes H|1\rangle = \frac{1}{\sqrt{2}} \sum_x |x\rangle \otimes |-\rangle$$

By phase kickback (established above), termwise, then summed:

$$\frac{1}{\sqrt{2}} \left[(-1)^{f(0)} |0\rangle + (-1)^{f(1)} |1\rangle \right] \otimes |-\rangle$$

Why it works

Write $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$. After the first two Hadamards, held ready for the oracle:

$$H|0\rangle \otimes H|1\rangle = \frac{1}{\sqrt{2}} \sum_x |x\rangle \otimes |-\rangle$$

By phase kickback (established above), termwise, then summed:

$$\frac{1}{\sqrt{2}} \left[(-1)^{f(0)} |0\rangle + (-1)^{f(1)} |1\rangle \right] \otimes |-\rangle$$

Apply H to x and look at the bracket:

- **Constant** ($f(0) = f(1)$): bracket is $\pm \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \rightarrow H \rightarrow \pm |0\rangle$. Measure **0**, always.
- **Balanced** ($f(0) \neq f(1)$): bracket is $\pm \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \rightarrow H \rightarrow \pm |1\rangle$. Measure **1**, always.

Why it works

Write $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$. After the first two Hadamards, held ready for the oracle:

$$H|0\rangle \otimes H|1\rangle = \frac{1}{\sqrt{2}} \sum_x |x\rangle \otimes |-\rangle$$

By phase kickback (established above), termwise, then summed:

$$\frac{1}{\sqrt{2}} \left[(-1)^{f(0)} |0\rangle + (-1)^{f(1)} |1\rangle \right] \otimes |-\rangle$$

Apply H to x and look at the bracket:

- **Constant** ($f(0) = f(1)$): bracket is $\pm \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \rightarrow H \rightarrow \pm |0\rangle$. Measure **0**, always.
- **Balanced** ($f(0) \neq f(1)$): bracket is $\pm \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \rightarrow H \rightarrow \pm |1\rangle$. Measure **1**, always.

One query, zero error, one classical query saved. Next: an *exponential* number saved.

Part 3

The Hadamard Transform

H on many qubits

On n qubits, apply H to each wire independently: $H^{\otimes n} = H \otimes H \otimes \cdots \otimes H$.

Example, $n = 2$, starting from $|00\rangle$:

$$H|0\rangle \otimes H|0\rangle = \frac{1}{2}(|0\rangle + |1\rangle) \otimes (|0\rangle + |1\rangle) = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

H on many qubits

On n qubits, apply H to each wire independently: $H^{\otimes n} = H \otimes H \otimes \cdots \otimes H$.

Example, $n = 2$, starting from $|00\rangle$:

$$H|0\rangle \otimes H|0\rangle = \frac{1}{2}(|0\rangle + |1\rangle) \otimes (|0\rangle + |1\rangle) = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

In general, starting from all zeros gives a uniform superposition over every n -bit string:

$$H^{\otimes n} |0\rangle^n = \frac{1}{\sqrt{2^n}} \sum_x |x\rangle$$

This is exactly the state-prep step both algorithms in this lecture use.

Starting somewhere else

What if the input isn't all zeros? Look at a single qubit in state $|x_i\rangle$:

- $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$: coefficient $+\frac{1}{\sqrt{2}}$ on *both* outputs.
- $H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$: coefficient $+\frac{1}{\sqrt{2}}$ on $|0\rangle$, $-\frac{1}{\sqrt{2}}$ on $|1\rangle$.

Both lines are one formula: the coefficient of $|z_i\rangle$ is $\frac{1}{\sqrt{2}}(-1)^{x_i z_i}$.

Starting somewhere else

What if the input isn't all zeros? Look at a single qubit in state $|x_i\rangle$:

- $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$: coefficient $+\frac{1}{\sqrt{2}}$ on *both* outputs.
- $H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$: coefficient $+\frac{1}{\sqrt{2}}$ on $|0\rangle$, $-\frac{1}{\sqrt{2}}$ on $|1\rangle$.

Both lines are one formula: the coefficient of $|z_i\rangle$ is $\frac{1}{\sqrt{2}}(-1)^{x_i z_i}$.

Multiplying independently across all n qubits, the signs multiply too:

$$H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_z (-1)^{x \cdot z} |z\rangle, \quad x \cdot z := x_1 z_1 \oplus x_2 z_2 \oplus \cdots \oplus x_n z_n$$

Starting somewhere else

What if the input isn't all zeros? Look at a single qubit in state $|x_i\rangle$:

- $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$: coefficient $+\frac{1}{\sqrt{2}}$ on *both* outputs.
- $H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$: coefficient $+\frac{1}{\sqrt{2}}$ on $|0\rangle$, $-\frac{1}{\sqrt{2}}$ on $|1\rangle$.

Both lines are one formula: the coefficient of $|z_i\rangle$ is $\frac{1}{\sqrt{2}}(-1)^{x_i z_i}$.

Multiplying independently across all n qubits, the signs multiply too:

$$H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_z (-1)^{x \cdot z} |z\rangle, \quad x \cdot z := x_1 z_1 \oplus x_2 z_2 \oplus \cdots \oplus x_n z_n$$

Check, $n = 2$, $x = 10$: $H|1\rangle \otimes H|0\rangle = \frac{1}{2}(|0\rangle - |1\rangle) \otimes (|0\rangle + |1\rangle) = \frac{1}{2}(|00\rangle + |01\rangle - |10\rangle - |11\rangle)$.

Matches: $x \cdot z$ is 0 for $z = 00, 01$ and 1 for $z = 10, 11$.

Starting somewhere else

What if the input isn't all zeros? Look at a single qubit in state $|x_i\rangle$:

- $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$: coefficient $+\frac{1}{\sqrt{2}}$ on *both* outputs.
- $H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$: coefficient $+\frac{1}{\sqrt{2}}$ on $|0\rangle$, $-\frac{1}{\sqrt{2}}$ on $|1\rangle$.

Both lines are one formula: the coefficient of $|z_i\rangle$ is $\frac{1}{\sqrt{2}}(-1)^{x_i z_i}$.

Multiplying independently across all n qubits, the signs multiply too:

$$H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_z (-1)^{x \cdot z} |z\rangle, \quad x \cdot z := x_1 z_1 \oplus x_2 z_2 \oplus \cdots \oplus x_n z_n$$

Check, $n = 2$, $x = 10$: $H|1\rangle \otimes H|0\rangle = \frac{1}{2}(|0\rangle - |1\rangle) \otimes (|0\rangle + |1\rangle) = \frac{1}{2}(|00\rangle + |01\rangle - |10\rangle - |11\rangle)$.
Matches: $x \cdot z$ is 0 for $z = 00, 01$ and 1 for $z = 10, 11$.

The one case Deutsch–Jozsa needs: set $z = 0^n$. Then $x \cdot z = 0$ for *every* x , so every sign is $+1$. The amplitude of 0^n is just $\frac{1}{\sqrt{2^n}}$ times the sum of whatever coefficients you started with, no signs to track.

Part 4

Deutsch–Jozsa

The problem, generalized

$f : \{0, 1\}^n \rightarrow \{0, 1\}$ is promised to be either

- **constant** (same value on all 2^n inputs), or
- **balanced** (exactly half of all 2^n inputs map to 1),

Decide which.

The problem, generalized

$f : \{0, 1\}^n \rightarrow \{0, 1\}$ is promised to be either

- **constant** (same value on all 2^n inputs), or
- **balanced** (exactly half of all 2^n inputs map to 1),

Decide which.

- **Classically:** worst case, $2^{n-1} + 1$ queries. An adversary answers 0 to your first 2^{n-1} queries; f could still be constant, or the very next input could flip it to balanced. Exponential in n .

The problem, generalized

$f : \{0, 1\}^n \rightarrow \{0, 1\}$ is promised to be either

- **constant** (same value on all 2^n inputs), or
- **balanced** (exactly half of all 2^n inputs map to 1),

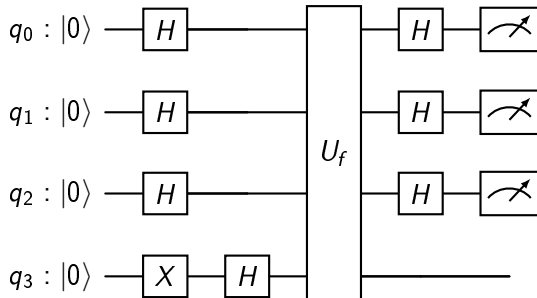
Decide which.

- **Classically:** worst case, $2^{n-1} + 1$ queries. An adversary answers 0 to your first 2^{n-1} queries; f could still be constant, or the very next input could flip it to balanced. Exponential in n .
- **Quantumly:** exactly 1 query, every time, for every n .

The circuit

Same shape as Deutsch's algorithm, scaled up: n query qubits plus one target, one oracle call, $U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$, $x \in \{0, 1\}^n$.

Shown here for $n = 3$: U_f is still a single black box, whatever f is.



Why it works: the oracle step

Same idea, n qubits. After Hadamards on the query register (all start $|0\rangle$) and on the target (start $|1\rangle$, so $H|1\rangle = |-\rangle$):

$$H^{\otimes n} |0\rangle^n \otimes |-\rangle = \frac{1}{\sqrt{2^n}} \sum_x |x\rangle \otimes |-\rangle \quad (x \text{ ranges over all } n\text{-bit strings})$$

Why it works: the oracle step

Same idea, n qubits. After Hadamards on the query register (all start $|0\rangle$) and on the target (start $|1\rangle$, so $H|1\rangle = |-\rangle$):

$$H^{\otimes n} |0\rangle^n \otimes |-\rangle = \frac{1}{\sqrt{2^n}} \sum_x |x\rangle \otimes |-\rangle \quad (x \text{ ranges over all } n\text{-bit strings})$$

Apply U_f to a single term $|x\rangle |-\rangle$, exactly as on Deutsch's slide, casing on $f(x) = 0$ or $f(x) = 1$:

$$U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle$$

Why it works: the oracle step

Same idea, n qubits. After Hadamards on the query register (all start $|0\rangle$) and on the target (start $|1\rangle$, so $H|1\rangle = |-\rangle$):

$$H^{\otimes n} |0\rangle^n \otimes |-\rangle = \frac{1}{\sqrt{2^n}} \sum_x |x\rangle \otimes |-\rangle \quad (x \text{ ranges over all } n\text{-bit strings})$$

Apply U_f to a single term $|x\rangle |-\rangle$, exactly as on Deutsch's slide, casing on $f(x) = 0$ or $f(x) = 1$:

$$U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle$$

So after the oracle, summing over every term:

$$\frac{1}{\sqrt{2^n}} \sum_x (-1)^{f(x)} |x\rangle \otimes |-\rangle$$

Why it works: reading off the answer

Going into the last step, the state is

$$\frac{1}{\sqrt{2^n}} \sum_x (-1)^{f(x)} |x\rangle$$

Apply $H^{\otimes n}$ one more time. We only care about the amplitude of 0^n , and by Part 3 that means: no new signs, just one more factor of $\frac{1}{\sqrt{2^n}}$:

$$\text{amp}(0^n) = \frac{1}{2^n} \sum_x (-1)^{f(x)}$$

Why it works: reading off the answer

Going into the last step, the state is

$$\frac{1}{\sqrt{2^n}} \sum_x (-1)^{f(x)} |x\rangle$$

Apply $H^{\otimes n}$ one more time. We only care about the amplitude of 0^n , and by Part 3 that means: no new signs, just one more factor of $\frac{1}{\sqrt{2^n}}$:

$$\text{amp}(0^n) = \frac{1}{2^n} \sum_x (-1)^{f(x)}$$

- **Constant:** every term has the same sign, so $\text{amp} = \pm 1$ and everything else is 0. Measure 0^n , always.
- **Balanced:** half the terms are $+1$, half are -1 , so $\text{amp} = 0$ exactly. Measuring 0^n is impossible; measure something else, always.

Why it works: reading off the answer

Going into the last step, the state is

$$\frac{1}{\sqrt{2^n}} \sum_x (-1)^{f(x)} |x\rangle$$

Apply $H^{\otimes n}$ one more time. We only care about the amplitude of 0^n , and by Part 3 that means: no new signs, just one more factor of $\frac{1}{\sqrt{2^n}}$:

$$\text{amp}(0^n) = \frac{1}{2^n} \sum_x (-1)^{f(x)}$$

- **Constant:** every term has the same sign, so $\text{amp} = \pm 1$ and everything else is 0. Measure 0^n , always.
- **Balanced:** half the terms are $+1$, half are -1 , so $\text{amp} = 0$ exactly. Measuring 0^n is impossible; measure something else, always.

One query, any n . Same argument as Deutsch's, one Hadamard layer wider.

Part 5

Programming with Qiskit

Setting up Qiskit

IBM's open-source Python SDK for building, transpiling, simulating and running quantum circuits, the same toolkit IBM's hardware stack runs on.

Local install, free, no account:

```
pip install qiskit qiskit-aer
```

Setting up Qiskit

IBM's open-source Python SDK for building, transpiling, simulating and running quantum circuits, the same toolkit IBM's hardware stack runs on.

Local install, free, no account:

```
pip install qiskit qiskit-aer
```

Gets you circuit-building plus AerSimulator: a fast classical simulator. Everything in the payoff section runs entirely on this: zero cost, zero queue, unlimited shots. Use it for all homework unless a problem asks for real hardware.

Real IBM hardware

Needs one more package, `qiskit-ibm-runtime`, plus an IBM Quantum Platform account. Circuits become jobs: they queue, then come back with real (noisy) statistics instead of the simulator's exact ones.

Two ways to get an account:

Real IBM hardware

Needs one more package, `qiskit-ibm-runtime`, plus an IBM Quantum Platform account. Circuits become jobs: they queue, then come back with real (noisy) statistics instead of the simulator's exact ones.

Two ways to get an account:

- An individual **Open Plan** account: free, limited monthly system time.

Real IBM hardware

Needs one more package, `qiskit-ibm-runtime`, plus an IBM Quantum Platform account. Circuits become jobs: they queue, then come back with real (noisy) statistics instead of the simulator's exact ones.

Two ways to get an account:

- An individual **Open Plan** account: free, limited monthly system time.
- A **Classroom Account**: I'll set one up for the course in the next week or two, ahead of the project release. IBM launched these this year for teaching: an instructor onboards the whole class at once (up to 100 students), no credit card for anyone, 10 minutes of real system time per student per month. That's the account you'll use for the project.

Real IBM hardware

Needs one more package, `qiskit-ibm-runtime`, plus an IBM Quantum Platform account. Circuits become jobs: they queue, then come back with real (noisy) statistics instead of the simulator's exact ones.

Two ways to get an account:

- An individual **Open Plan** account: free, limited monthly system time.
- A **Classroom Account**: I'll set one up for the course in the next week or two, ahead of the project release. IBM launched these this year for teaching: an instructor onboards the whole class at once (up to 100 students), no credit card for anyone, 10 minutes of real system time per student per month. That's the account you'll use for the project.

Workflow either way: **build** the circuit → **transpile** it (native gates, device connectivity) → **run** it, locally or as a submitted job. Only the last step changes.

Qiskit toolkit reference

Call	What it does
<code>QuantumCircuit(n,m)</code>	n qubits (start at $ 0\rangle$), m classical bits
<code>.h(q)</code> , <code>.x(q)</code> , <code>.z(q)</code>	single-qubit gates; one index or a range
<code>.cx(c,t)</code> , <code>.ccx(c1,c2,t)</code>	CNOT, Toffoli
<code>.barrier()</code>	visual/scheduling divider only
<code>.compose(other, inplace=True)</code>	splice one circuit into another
<code>.measure(qubits, clbits)</code>	measure into classical bits
<code>.draw(output="text")</code>	ASCII-art the circuit

Qiskit toolkit reference

Call	What it does
<code>QuantumCircuit(n,m)</code>	n qubits (start at $ 0\rangle$), m classical bits
<code>.h(q)</code> , <code>.x(q)</code> , <code>.z(q)</code>	single-qubit gates; one index or a range
<code>.cx(c,t)</code> , <code>.ccx(c1,c2,t)</code>	CNOT, Toffoli
<code>.barrier()</code>	visual/scheduling divider only
<code>.compose(other, inplace=True)</code>	splice one circuit into another
<code>.measure(qubits, clbits)</code>	measure into classical bits
<code>.draw(output="text")</code>	ASCII-art the circuit

Call	What it does
<code>transpile(qc, backend)</code>	rewrite into a device's native gates/layout
<code>AerSimulator()</code>	local, free, exact simulator backend
<code>sim.run(qc, shots=n).result()</code>	execute, return counts
<code>QiskitRuntimeService()</code> , <code>.least_busy()</code>	connect to a real backend
<code>SamplerV2</code>	run on real hardware, return statistics
<code>plot_histogram(counts)</code>	bar chart of outcomes

Part 6

The Payoff

Building the oracles

An oracle builder for constant f , one for balanced f . Neither one knows it's being tested.

```
def constant_oracle(n, value=0):
    qc = QuantumCircuit(n + 1, name="Oracle")
    if value == 1:
        qc.x(n)          #  $f(x) = 1$  for every  $x$ 
    return qc             #  $f(x) = 0$  for every  $x$ : do nothing

def balanced_oracle(n):
    qc = QuantumCircuit(n + 1, name="Oracle")
    for i in range(n):
        qc.cx(i, n)      #  $f(x) = x_1 \text{ XOR } \dots \text{ XOR } x_n$ 
    return qc
```

The Deutsch–Jozsa circuit

Takes whichever oracle you hand it, and doesn't care which kind it is. That's the point.

```
def deutsch_jozsa_circuit(n, oracle):  
    qc = QuantumCircuit(n + 1, n)      # n query qubits + 1 target  
    qc.x(n)                             # target -> |1>  
    qc.h(range(n + 1))                  # Hadamard everything  
    qc.barrier()  
    qc.compose(oracle, inplace=True)  
    qc.barrier()  
    qc.h(range(n))                       # Hadamard the query register again  
    qc.measure(range(n), range(n))  
    return qc
```

Running it

```
n = 3
sim = AerSimulator()
for label, oracle in [("constant", constant_oracle(n, 1)),
                      ("balanced", balanced_oracle(n))]:
    qc = deutsch_jozsa_circuit(n, oracle)
    tqc = transpile(qc, sim)
    counts = sim.run(tqc, shots=1024).result().get_counts()
    print(label, counts)
```

Running it

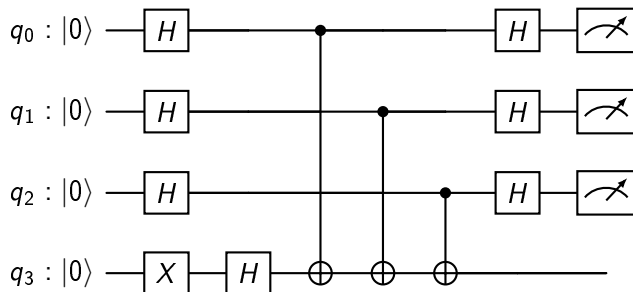
```
n = 3
sim = AerSimulator()
for label, oracle in [("constant", constant_oracle(n, 1)),
                      ("balanced", balanced_oracle(n))]:
    qc = deutsch_jozsa_circuit(n, oracle)
    tqc = transpile(qc, sim)
    counts = sim.run(tqc, shots=1024).result().get_counts()
    print(label, counts)
```

constant {'000': 1024}

balanced {'111': 1024}

Reading the result

This is `deutsch_jozsa_circuit(3, balanced_oracle(3))` with U_f unpacked into the three CNOTs that actually implement it. The general circuit shown earlier had this whole block inside one U_f box.



Exactly the theory from earlier: constant returns 000 on all 1024 shots, balanced returns a fixed nonzero string on all 1024 shots. No overlap, no error rate to argue about. Read off constant vs. balanced from one circuit, one oracle call, run once.

On real hardware

The circuit-building code doesn't change. Only the last step does:

```
service = QiskitRuntimeService()
backend = service.least_busy(operational=True, simulator=False)
pm = generate_preset_pass_manager(target=backend.target,
                                  optimization_level=1)
isa_qc = pm.run(deutsch_jozsa_circuit(n, balanced_oracle(n)))
result = SamplerV2(mode=backend).run([isa_qc], shots=1024).result()
counts = result[0].data.c.get_counts()
```

On real hardware

The circuit-building code doesn't change. Only the last step does:

```
service = QiskitRuntimeService()
backend = service.least_busy(operational=True, simulator=False)
pm = generate_preset_pass_manager(target=backend.target,
                                  optimization_level=1)
isa_qc = pm.run(deutsch_jozsa_circuit(n, balanced_oracle(n)))
result = SamplerV2(mode=backend).run([isa_qc], shots=1024).result()
counts = result[0].data.c.get_counts()
```

Real devices are noisy: expect 111 to *dominate*, not own every shot. Deciding “balanced” from a most-likely outcome instead of a certain one is worth thinking about before the project.

Recap

- Oracles are XOR-embeddings, $U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$: always unitary, for free, for any f .

Recap

- Oracles are XOR-embeddings, $U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$: always unitary, for free, for any f .
- Both algorithms run on one fact, phase kickback: $U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle$.

Recap

- Oracles are XOR-embeddings, $U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$: always unitary, for free, for any f .
- Both algorithms run on one fact, phase kickback: $U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle$.
- Deutsch's algorithm: one query decides constant vs. balanced on $\{0, 1\}$, where classical needs two.

Recap

- Oracles are XOR-embeddings, $U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$: always unitary, for free, for any f .
- Both algorithms run on one fact, phase kickback: $U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle$.
- Deutsch's algorithm: one query decides constant vs. balanced on $\{0, 1\}$, where classical needs two.
- Deutsch-Jozsa: the same one query, on $\{0, 1\}^n$, where classical needs up to $2^{n-1} + 1$: exponential.

Recap

- Oracles are XOR-embeddings, $U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$: always unitary, for free, for any f .
- Both algorithms run on one fact, phase kickback: $U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle$.
- Deutsch's algorithm: one query decides constant vs. balanced on $\{0, 1\}$, where classical needs two.
- Deutsch-Jozsa: the same one query, on $\{0, 1\}^n$, where classical needs up to $2^{n-1} + 1$: exponential.
- Qiskit: free local simulation now; real IBM hardware via Classroom Accounts for the project.